Практическая работа №5

Тестовая документация проекта SkillLvlUp

1. **Введение**

Настоящий документ является отчётной документацией по практической работе №5 и включает в себя тест-план, тест-кейсы, баг-репорты, а также результаты согласования и финализации документации по проекту SkillLvlUp.

SkillLvlUp — это web-приложение (LMS), разработанное на стеке: HTML, CSS, JavaScript, Node.js (Express) с использованием MySQL. Приложение предоставляет пользователям возможность самостоятельного управления обучением: создавать учебные курсы, уроки, отслеживать прогресс, загружать файлы, вести заметки и формировать отчёты.

2. **Тест-план**

2.1. **Цели и задачи тестирования**

Цель тестирования — проверка корректности функционирования всех основных модулей SPA-приложения SkillLvlUp в соответствии с функциональными требованиями.

Задачи:

- Проверить сценарии авторизации (регистрация, вход, выход).
- Протестировать CRUD-операции для курсов и уроков.
- Проверить работу модуля управления файлами и заметками.
- Протестировать дашборд и систему отслеживания прогресса.
- Проверить формирование отчётов (XLSX, PDF) и построение графиков.
- Проверить работу календаря и системы дедлайнов.
- Зафиксировать обнаруженные дефекты и подготовить баг-репорты.

2.2**. Объекты тестирования**

№	Модуль	Описание
1	Авторизация	Регистрация нового пользователя, вход/выход из системы, обновление профиля
2	Курсы	Создание, просмотр, редактирование, удаление учебных планов; поиск и фильтрация
3	Уроки	CRUD-операции над уроками в рамках курса; отметка о выполнении, заметки
4	Файлы и материалы	Загрузка, скачивание, удаление файлов уроков

5	Дашборд	Отображение виджетов статистики, активных курсов, календаря и списка дедлайнов
6	Отчёты	График активности, распределение времени, выгрузка XLSX/PDF, таблица статистики
7	Календарь	Навигация по месяцам, отображение событий (дедлайнов и уроков) на датах

2.3. **Виды тестирования**

Вид тестирования	Применение
Функциональное	Проверка соответствия реализованных функций основным пользовательским сценариям
Негативное	Ввод некорректных данных, проверка граничных условий и обработки ошибок
Дымовое	Быстрая проверка запуска приложения и доступности ключевых разделов
Регрессионное	Повторное тестирование после исправления дефектов
Ручное (чёрный ящик)	Основной метод: тестировщик взаимодействует с UI без доступа к коду

2.4. **Приоритеты сценариев**

Модуль	Сценарий	Приоритет
Авторизация	Регистрация с валидными данными	Высокий
Авторизация	Регистрация с невалидными данными	Высокий
Авторизация	Вход в систему	Высокий
Авторизация	Выход из системы	Средний
Курсы	Создание курса	Высокий
Курсы	Редактирование курса	Высокий
Курсы	Удаление курса	Средний
Курсы	Поиск и фильтрация	Средний
Уроки	Создание / редактирование / удаление	Средний
Уроки	Отметка о выполнении (прогресс)	Высокий
Дашборд	Отображение статистики и календаря	Средний
Отчёты	Выгрузка XLSX и PDF	Низкий
Настройки	Обновление профиля	Низкий

2.5. **Критерии начала и завершения тестирования**

Критерии начала:

- Проект развёрнут локально (Live Server).
- База данных MySQL инициализирована и содержит тестовые данные.

Критерии завершения:

- Все тест-кейсы выполнены.
- Найденные дефекты зафиксированы в баг-репортах.
- Критические и высокоприоритетные баги исправлены и прошли повторное тестирование.

2.6. **Тестовая среда**

Параметр	Значение
ОС	Windows 10
Браузер	Google Chrome
Frontend	HTML5, CSS3, JavaScript
Backend	Node.js, Express
База Данных	MySQL
Метод тестирования	Ручное (чёрный ящик)

3. **Тест-кейсы**

3.1. **Модуль «Авторизация»**

ТК-01. Регистрация с валидными данными
Тип тестированияРучное / Позитивное / Функциональное

Предусловие: Пользователь не зарегистрирован. Приложение открыто в браузере.

Шаг	Действие	Ожидаемый результат/статус
1	Открыть приложение.	Загружается форма входа. **[Пройден]**
2	Нажать "Зарегистрироваться".	Открывается форма регистрации. **[Пройден]**
3	Ввести: Имя "Иван Иванов", Email "test@mail.ru", Пароль "Qwerty1!"	Данные вводятся без ошибок. **[Пройден]**
4	Нажать "Создать аккаунт".	Появляется уведомление об успехе; открывается главная страница (дашборд). **[Пройден]**

ТК-02. Регистрация с невалидным email

Тип тестирования Ручное / Негативное / Функциональное

Предусловие: Открыта страница регистрации.

Шаг	Действие	Ожидаемый результат/статус
1	Ввести некорректный email: "notanemail".	Поле заполнено. **[Пройден]**
2	Ввести корректные имя и пароль.	Поля заполнены. **[Пройден]**
3	Нажать "Создать аккаунт".	**Ожидается:** Сообщение "Неверный формат email". **Фактически:** Сообщение получено. **[Пройден]**

ТК-03. Вход в систему с корректными данными

Тип тестирования Ручное / Позитивное / Функциональное

Предусловие: Пользователь зарегистрирован.

Шаг	Действие	Ожидаемый результат/статус
1	Открыть страницу входа.	Форма входа загружена. **[Пройден]**
2	Ввести корректные email и пароль.	Данные введены. **[Пройден]**
3	Нажать "Войти".	Успешный вход, загружается дашборд с именем пользователя. **[Пройден]**

ТК-04. Вход с неверным паролем

Тип тестирования Ручное / Негативное / Функциональное

Предусловие: Открыта страница входа.

Шаг	Действие	Ожидаемый результат/статус
1	Ввести корректный email.	Поле заполнено. **[Пройден]**
2	Ввести неверный пароль: "wrongPass99".	Поле заполнено. **[Пройден]**
3	Нажать "Войти".	Сообщение "Неверный email или пароль". **[Пройден]**

ТК-05. Выход из системы

Тип тестирования: Ручное / Позитивное / Функциональное

Предусловие: Пользователь авторизован и находится на дашборде.

Шаг	**Действие**	**Ожидаемый результат / статус**
1	Нажать на аватар профиля в верхнем меню.	Открывается выпадающее меню. [Пройден]
2	Выбрать пункт «Выйти».	Пользователь перенаправлен на страницу входа. [Пройден]
3	Нажать кнопку «Назад» в браузере.	Страница дашборда не загружается, требуется повторный вход. [Пройден]

ТК-06. Обновление профиля пользователя

Тип тестирования: Ручное / Позитивное / Функциональное

Предусловие: Пользователь авторизован.

Шаг	**Действие**	**Ожидаемый результат / статус**
1	Перейти в раздел «Настройки» / «Профиль».	Открывается форма редактирования профиля. [Пройден]
2	Изменить поле «Имя» на «Петр Петров».	Данные введены в поле. [Пройден]
3	Нажать «Сохранить».	Появляется уведомление об успехе, имя обновлено в шапке сайта. [Пройден]

3.2. Модуль «Курсы»

ТК-07. Создание курса с валидными данными

Тип тестирования Ручное / Позитивное / Функциональное

Предусловие: Пользователь авторизован.

Шаг	Действие	Ожидаемый результат/статус
1	Перейти в раздел "Мои курсы".	Отображается список курсов. **[Пройден]**
2	Нажать "Новый курс".	Появляется модальное окно. **[Пройден]**
3	Заполнить поля: Название "Курс JS", Описание "Изучаем JS", Даты.	Данные введены. **[Пройден]**
4	Нажать "Сохранить".	Модальное окно закрывается, курс появляется в списке. **[Пройден]**

ТК-08. Создание курса с пустым названием

Тип тестирования Ручное / Негативное / Функциональное

Предусловие: Открыто модальное окно создания курса.

Шаг	Действие	Ожидаемый результат/статус
1	Оставить поле "Название" пустым.	Поле не заполнено. **[Пройден]**
2	Заполнить остальные поля.	Данные введены. **[Пройден]**
3	Нажать "Сохранить".	**Фактически:** Запрос уходит, сервер возвращает ошибку 400. **[Провален]**

ТК-09. Редактирование существующего курса

Тип тестирования: Ручное / Позитивное / Функциональное

Предусловие: В системе создан минимум один курс.

Шаг	Действие	Ожидаемый результат / статус
1	В списке курсов нажать кнопку «Редактировать» у курса.	Открывается модальное окно с текущими данными курса. [Пройден]
2	Изменить описание курса.	Текст в поле изменен. [Пройден]
3	Нажать «Сохранить».	Модальное окно закрывается, обновленное описание отображается в списке. [Пройден]

ТК-10. Удаление курса

Тип тестирования: Ручное / Позитивное / Функциональное

Предусловие: В системе создан курс, который не нужен.

Шаг	Действие	Ожидаемый результат / статус
1	Нажать кнопку «Удалить» у курса.	Появляется системное подтверждение действия. [Пройден]
2	Подтвердить удаление.	Курс исчезает из списка. [Пройден]
3	Обновить страницу.	Удаленный курс не появляется повторно. [Пройден]

ТК-11. Поиск курса по названию

Тип тестирования: Ручное / Позитивное / Функциональное

Предусловие: В системе создано несколько курсов.

Шаг	Действие	Ожидаемый результат / статус
1	В поле поиска ввести часть названия курса (например, "JS").	Список фильтруется в реальном времени или по нажатию. [Пройден]
2	Проверить результаты.	Отображаются только курсы, содержащие "JS" в названии. [Пройден]
3	Очистить поле поиска.	Список курсов восстанавливается в полном объеме. [Пройден]

3.3. Модуль «Уроки»

ТК-12. Отметка урока как выполненного (прогресс)

Тип тестирования Ручное / Позитивное / Функциональное

Предусловие: В системе есть курс с минимум одним уроком.

Шаг	Действие	Ожидаемый результат/статус
1	Открыть детальную страницу курса.	Отображается список уроков. **[Пройден]**
2	Нажать чекбокс у урока.	Анимация выполнения, прогресс-бар курса пересчитывается. **[Пройден]**
3	Перейти на дашборд.	Счётчик "Уроков пройдено" увеличился. **[Пройден]**

ТК-13. Создание урока внутри курса

Тип тестирования: Ручное / Позитивное / Функциональное

Предусловие: Открыта детальная страница существующего курса.

Шаг	**Действие**	**Ожидаемый результат / статус**
1	Нажать кнопку «Добавить урок».	Открывается форма создания урока. [Пройден]
2	Заполнить название и дату дедлайна.	Данные введены. [Пройден]
3	Нажать «Сохранить».	Урок появляется в списке уроков курса. [Пройден]

ТК-14. Редактирование урока

Тип тестирования: Ручное / Позитивное / Функциональное

Предусловие: В курсе есть минимум один урок.

Шаг	**Действие**	**Ожидаемый результат / статус**
1	Нажать на иконку редактирования у урока.	Открывается форма редактирования. [Пройден]
2	Изменить дату дедлайна.	Дата изменена. [Пройден]
3	Нажать «Сохранить».	Новая дата отображается в списке уроков. [Пройден]

ТК-15. Удаление урока

Тип тестирования: Ручное / Позитивное / Функциональное

Предусловие: В курсе есть урок.

Шаг	Действие	Ожидаемый результат / статус
1	Нажать кнопку «Удалить» у урока.	Появляется запрос подтверждения. [Пройден]
2	Подтвердить удаление.	Урок удаляется из списка. [Пройден]
3	Проверить прогресс курса.	Прогресс-бар пересчитывается без учета удаленного урока. [Пройден]

ТК-16. Добавление заметки к уроку

Тип тестирования: Ручное / Позитивное / Функциональное

Предусловие: Открыта страница урока.

Шаг	Действие	Ожидаемый результат / статус
1	В поле «Заметки» ввести текст «Важная формула».	Текст введен. [Пройден]
2	Нажать «Сохранить заметку».	Появляется уведомление об успехе. [Пройден]
3	Обновить страницу урока.	Текст заметки сохраняется и отображается. [Пройден]

ТК-17. Загрузка файла к уроку

Тип тестирования: Ручное / Позитивное / Функциональное

Предусловие: Открыта страница урока, режим редактирования.

Шаг	Действие	Ожидаемый результат / статус
1	Нажать «Прикрепить файл».	Открывается диалог выбора файла ОС. [Пройден]
2	Выбрать файл формата .pdf (менее 5 Мб).	Файл выбран. [Пройден]
3	Сохранить урок.	Файл отображается в списке материалов урока. [Пройден]

ТК-18. Скачивание файла материала

Тип тестирования: Ручное / Позитивное / Функциональное

Предусловие: У урока есть прикрепленный файл.

Шаг	Действие	Ожидаемый результат / статус
1	Нажать на название файла в списке материалов.	Начинается загрузка файла браузером. [Пройден]
2	Открыть загруженный файл.	Содержимое файла соответствует загруженному. [Пройден]

ТК-19. Удаление файла материала

Тип тестирования: Ручное / Позитивное / Функциональное

Предусловие: У урока есть прикрепленный файл.

Шаг	Действие	Ожидаемый результат / статус
1	Нажать иконку «Корзина» рядом с файлом.	Файл исчезает из списка материалов. [Пройден]
2	Обновить страницу.	Файл не восстанавливается. [Пройден]

ТК-20. Загрузка файла превышающего лимит (Негативный)

Тип тестирования: Ручное / Негативное / Функциональное

Предусловие: Открыта форма загрузки файла. Лимит сервера 5 Мб.

Шаг	Действие	Ожидаемый результат / статус
1	Выбрать файл размером 10 Мб.	Файл выбран в диалоге. [Пройден]
2	Попытаться сохранить урок.	Появляется сообщение «Файл слишком большой». Запрос на сервер не уходит. [Пройден]

ТК-21. Выгрузка отчёта в формате PDF

Тип тестирования Ручное / Позитивное / Функциональное

Предусловие: В системе есть курсы.

Шаг	Действие	Ожидаемый результат/статус
1	Перейти в раздел "Отчеты".	Отображаются графики и таблица. [Пройден]
2	Нажать "Выгрузка PDF".	Начинается скачивание файла. [Пройден]
3	Открыть скачанный PDF	Документ содержит имя пользователя и таблицу статистики. [Пройден]

ТК-22. Выгрузка отчёта в формате XLSX

Тип тестирования Ручное / Позитивное / Функциональное

Предусловие: В системе есть курсы и уроки.

Шаг	Действие	Ожидаемый результат/статус
1	Перейти в раздел "Отчеты".Отображаются графики и таблица	Отображаются графики и таблица. **[Пройден]**
2	Нажать "Выгрузка XLSX".Начинается скачивание файла	Начинается скачивание файла. **[Пройден]**
3	Открыть файл в Excel/LibreOffice	Документ содержит 3 листа: Общая статистика, Курсы, Уроки. **[Пройден]**

3.4 Модуль «Дашборд»

ТК-23. Отображение виджета общей статистики

Тип тестирования: Ручное / Позитивное / Функциональное

Предусловие: Пользователь авторизован, есть пройденные уроки.

Шаг	**Действие**	**Ожидаемый результат / статус**
1	Открыть главную страницу (Дашборд).	Виджеты загружены. [Пройден]
2	Проверить виджет «Всего пройдено».	Число соответствует сумме пройденных уроков во всех курсах. [Пройден]
3	Проверить виджет «Активные курсы».	Количество соответствует курсам со статусом «В процессе». [Пройден]

ТК-24. Отображение списка ближайших дедлайнов

Тип тестирования: Ручное / Позитивное / Функциональное

Предусловие: В курсах есть уроки с датами дедлайнов.

Шаг	Действие	Ожидаемый результат / статус
1	Открыть Дашборд.	Виджет «Дедлайны» отображается. [Пройден]
2	Проверить список.	Отображаются уроки с ближайшими датами сдачи. [Пройден]
3	Нажать на элемент дедлайна.	Происходит переход к соответствующему уроку в курсе. [Пройден]

3.5 Модуль «Календарь»

ТК-25. Навигация по месяцам в календаре

Тип тестирования: Ручное / Позитивное / Функциональное

Предусловие: Открыт раздел «Календарь» на Дашборде.

Шаг	Действие	Ожидаемый результат / статус
1	Нажать стрелку «Вперед» (следующий месяц).	Отображение календаря переключается на следующий месяц. [Пройден]
2	Нажать стрелку «Назад» (предыдущий месяц).	Отображение календаря возвращается на текущий месяц. [Пройден]
3	Проверить сохранение событий.	Маркеры событий (дедлайнов) отображаются на корректных датах. [Пройден]

4. Баг-репорты

4.1. Обнаруженные дефекты

БР-01: Кнопка "Сохранить" активна при создании курса без обязательного поля "Название"

Описание проблемы	При создании нового курса можно нажать кнопку "Сохранить", даже если поле "Название курса" не заполнено. Атрибут required у поля в HTML присутствует, но JS-обработчик формы вызывает API напрямую, минуя стандартную браузерную валидацию из-за использования e.preventDefault(). Это приводит к запросу на сервер с пустым названием и ошибке сервера.
Серьёзность (Severity)	Средний (Major)
Приоритет (Priority)	Высокий
Статус	Исправлен
Шаги воспроизведения	1. Авторизоваться. 2. Нажать кнопку «Новый курс». 3. Оставить поле «Название курса» пустым. 4. Заполнить остальные поля. 5. Нажать кнопку «Сохранить».
Ожидаемый результат	Кнопка «Сохранить» неактивна или появляется сообщение «Заполните обязательные поля» без отправки запроса на сервер.
Фактический результат	Выполняется запрос к API, сервер возвращает ошибку валидации (400 Bad Request).

БР-02: Не полная локализация интерфейса при формировании PDF в Excel-подобном экспорте

Описание проблемы	При экспорте в Excel (XLSX) подписи колонок формируются на русском языке, однако заголовок листа "Общая статистика" может сломаться в старых версиях Excel из-за отсутствия BOM для кириллицы (зависит от версии библиотеки).
Серьёзность (Severity)	Незначительный (Minor)
Приоритет (Priority)	Низкий
Статус	Исправлен
Шаги воспроизведения	1. Перейти в раздел «Отчеты». 2. Нажать «Выгрузка XLSX». 3. Открыть файл в Excel 2010.
Ожидаемый результат	Все названия вкладок и заголовки корректно читаемы.
Фактический результат	В устаревших версиях Excel названия вкладок могут отображаться как "РћР±С‰Р°СЏ СЃС‚Р°С‚РёСЃС‚РёРєР°".

БР-03: Дублирование запросов при быстром переключении чекбокса урока (состояние гонки)

Описание проблемы	При очень быстром двойном клике по чекбоксу выполнения урока отправляется два PATCH-запроса на сервер. Первый запрос меняет статус на true, второй — обратно на false. Пользователь видит, что отметка «залипла» в неверном положении или урок не засчитался.
Серьёзность (Severity)	Средний (Major)
Приоритет (Priority)	Средний
Статус	Исправлен
Шаги воспроизведения	1. Открыть детали курса. 2. Очень быстро дважды кликнуть по чекбоксу урока.
Ожидаемый результат	Статус урока меняется однократно, прогресс пересчитывается корректно.
Фактический результат	Урок отмечается и сразу снимается, либо возникает рассинхронизация данных на фронте и бэке до перезагрузки страницы

БР-04: Ошибка сервера 500 при загрузке файла превышающего лимит

Параметр	Значение
Модуль	Файлы и материалы
Серьезность	Средняя (Major)
Приоритет	Высокий
Статус	Исправлен
Описание	Пользователь может выбрать файл размером 20 Мб (при лимите 5 Мб). Загрузка начинается, но сервер обрывает соединение с ошибкой 500, интерфейс зависает в состоянии «Загрузка».
Шаги воспроизведения	1. Открыть урок. 2. Прикрепить файл 20 Мб. 3. Нажать «Сохранить».

БР-05: Неверный расчет прогресса на дашборде после удаления курса

Параметр	Значение
Модуль	Дашборд / Курсы
Серьезность	Высокая (Critical)
Приоритет	Высокий
Статус	Исправлен
Описание	После удаления курса, в котором были пройденные уроки, виджет «Всего пройдено» на дашборде не обновляется. Сумма пройденных уроков становится больше, чем общее количество доступных уроков (процент > 100% или некорректное число).
Шаги воспроизведения	1. Пройти несколько уроков в курсе. 2. Запомнить статистику на дашборде. 3. Удалить курс. 4. Вернуться на дашборд.

Решения и повторное тестирование

БР-01: Кнопка "Сохранить" активна при создании курса без обязательного поля "Название"

Действия по исправлению	**Frontend (JS):** В обработчик события submit формы добавлена проверка значения поля перед отправкой запроса. `if (!courseTitle.trim()) { showValidationMessage(); return; }` Также добавлено визуальное подсвечивание поля красной рамкой.
Повторное тестирование	**ТК-06** (Создание курса с пустым названием).
Результат повторного тестирования	**[Пройден]** Кнопка «Сохранить» блокируется или появляется сообщение об ошибке до отправки запроса на сервер. Запрос в сеть не уходит.

БР-02: Не полная локализация интерфейса при формировании PDF в Excel-подобном экспорте

Действия по исправлению	**Backend (Node.js):** Библиотека генерации отчетов заменена на exceljs, которая корректно обрабатывает UTF-8 по умолчанию. Для совместимости добавлен явный заголовок BOM (\ufeff) в начало файла при потоковой передаче.
Повторное тестирование	**ТК-09** (Выгрузка отчёта в формате XLSX).
Результат повторного тестирования	**[Пройден]** Все названия вкладок и ячейки с русским текстом отображаются корректно во всех версиях Excel.

БР-03: Дублирование запросов при быстром переключении чекбокса урока (состояние гонки)

Действия по исправлению	**Frontend (JS):** Реализована блокировка элемента управления на время ожидания ответа от сервера. checkbox.disabled = true; внутри функции запроса. checkbox.disabled = false; в блоке finally. Добавлен индикатор загрузки (спиннер) рядом с чекбоксом.
Повторное тестирование	**ТК-07** (Отметка урока как выполненного) + стресс-тест кликами.
Результат повторного тестирования	**[Пройден]** Повторные клики игнорируются до получения ответа от сервера. Прогресс-бар обновляется корректно.

БР-04: Ошибка сервера 500 при загрузке файла превышающего лимит

Действия по исправлению	**Frontend (JS):** Добавлена проверка file.size перед инициированием загрузки. if (file.size > 5 * 1024 * 1024) { alert('Файл слишком большой'); return; } **Backend (Express):** Настроен middleware multer с явным ограничением limits: { fileSize: 5 * 1024 * 1024 } и обработкой ошибки LIMIT_FILE_SIZE.
Повторное тестирование	**ТК-22** (Загрузка файла превышающего лимит).
Результат повторного тестирования	**[Пройден]** Файл не отправляется на сервер. Пользователь мгновенно получает понятное уведомление об ошибке на клиенте.

БР-05: Неверный расчет прогресса на дашборде после удаления курса

Действия по исправлению	**Backend (Node.js/MySQL):** Логика удаления курса (DELETE FROM courses) дополнена каскадным удалением связанных уроков и обновлением сводной таблицы статистики пользователя. Или реализован пересчет статистики «на лету» через JOIN запрос вместо хранения кэшированного счетчика в таблице users. Выбран вариант пересчета через запрос COUNT(lessons.completed) при каждом запросе дашборда.
Повторное тестирование	**ТК-13** (Удаление курса) + **ТК-23** (Отображение виджета общей статистики).
Результат повторного тестирования	**[Пройден]** После удаления курса статистика на дашборде автоматически пересчитывается. Прогресс отображается корректно.

4. АВТОТЕСТЫ

AT-01:

```
const request = require('supertest');
const app = require('../../server');
const db = require('../../config/database');

describe('API Авторизация', () => {
  beforeAll(async () => {
    // Создание тестового пользователя
    await db.query(
      'INSERT INTO users (name, email, password) VALUES (?, ?, ?)',
      ['Тест Пользователь', 'autotest@mail.ru', 'hashedPassword123']
    );
  });

  afterAll(async () => {
    // Очистка тестовых данных
    await db.query('DELETE FROM users WHERE email = ?', ['autotest@mail.ru']);
    await db.end();
  });

  test('AT-01: Вход с корректными данными возвращает токен', async () => {
    const response = await request(app)
      .post('/api/auth/login')
      .send({
        email: 'autotest@mail.ru',
        password: 'hashedPassword123'
      });

    expect(response.status).toBe(200);
    expect(response.body).toHaveProperty('token');
    expect(response.body.user).toHaveProperty('id');
    expect(response.body.user.email).toBe('autotest@mail.ru');
  });
});
```

AT-02:

```
const request = require('supertest');
const app = require('../../server');
const db = require('../../config/database');

let authToken;

beforeAll(async () => {
  // Получение токена авторизации
  const loginRes = await request(app)
    .post('/api/auth/login')
```

```
    .send({ email: 'autotest@mail.ru', password: 'hashedPassword123' });
  authToken = loginRes.body.token;
});

describe('API Курсы', () => {
  test('AT-02: Создание курса с пустым названием возвращает 400', async () => {
    const response = await request(app)
      .post('/api/courses')
      .set('Authorization', `Bearer ${authToken}`)
      .send({
        title: '',
        description: 'Тестовое описание',
        startDate: '2025-01-01',
        endDate: '2025-12-31'
      });

    expect(response.status).toBe(400);
    expect(response.body).toHaveProperty('error');
    expect(response.body.error).toContain('Название курса обязательно');
  });

  test('AT-02.1: Создание курса с валидными данными возвращает 201', async () =>
{
    const response = await request(app)
      .post('/api/courses')
      .set('Authorization', `Bearer ${authToken}`)
      .send({
        title: 'Автотест Курс JS',
        description: 'Курс для автотестов',
        startDate: '2025-01-01',
        endDate: '2025-12-31'
      });

    expect(response.status).toBe(201);
    expect(response.body.course).toHaveProperty('id');
    expect(response.body.course.title).toBe('Автотест Курс JS');
  });
});
```

AT-03:

```
const request = require('supertest');
const app = require('../../server');
const db = require('../../config/database');

let authToken;
let lessonId;

beforeAll(async () => {
  const loginRes = await request(app)
    .post('/api/auth/login')
    .send({ email: 'autotest@mail.ru', password: 'hashedPassword123' });
  authToken = loginRes.body.token;

  // Создание тестового урока
  const courseRes = await request(app)
    .post('/api/courses')
    .set('Authorization', `Bearer ${authToken}`)
    .send({
      title: 'Курс для тестов уроков',
      description: 'Описание',
      startDate: '2025-01-01',
      endDate: '2025-12-31'
    });

  const lessonRes = await request(app)
    .post(`/api/courses/${courseRes.body.course.id}/lessons`)
    .set('Authorization', `Bearer ${authToken}`)
    .send({
      title: 'Урок 1',
      deadline: '2025-06-01'
    });

  lessonId = lessonRes.body.lesson.id;
});

describe('API Уроки', () => {
  test('AT-03: Отметка урока как выполненного обновляет прогресс', async () => {
    const response = await request(app)
      .patch(`/api/lessons/${lessonId}/complete`)
      .set('Authorization', `Bearer ${authToken}`)
      .send({ completed: true });

    expect(response.status).toBe(200);
    expect(response.body.lesson.completed).toBe(true);
    expect(response.body).toHaveProperty('courseProgress');
    expect(response.body.courseProgress).toBeGreaterThanOrEqual(0);
    expect(response.body.courseProgress).toBeLessThanOrEqual(100);
  });
});
```

AT-04:

```
/**
 * @jest-environment jsdom
 */

const { validateFileSize, validateFileType } =
require('../../public/js/utils/fileValidation');

describe('Frontend Валидация файлов', () => {
  const MAX_FILE_SIZE = 5 * 1024 * 1024; // 5 MB

  test('AT-04: Файл меньше лимита проходит валидацию', () => {
    const mockFile = {
      name: 'document.pdf',
      size: 2 * 1024 * 1024, // 2 MB
      type: 'application/pdf'
    };

    const result = validateFileSize(mockFile, MAX_FILE_SIZE);
    expect(result.valid).toBe(true);
    expect(result.error).toBeNull();
  });

  test('AT-04.1: Файл больше лимита не проходит валидацию', () => {
    const mockFile = {
      name: 'large-video.mp4',
      size: 10 * 1024 * 1024, // 10 MB
      type: 'video/mp4'
    };

    const result = validateFileSize(mockFile, MAX_FILE_SIZE);
    expect(result.valid).toBe(false);
    expect(result.error).toContain('Превышен максимальный размер файла');
  });

  test('AT-04.2: Недопустимый тип файла отклоняется', () => {
    const mockFile = {
      name: 'script.exe',
      size: 1 * 1024 * 1024,
      type: 'application/x-executable'
    };

    const allowedTypes = ['application/pdf', 'image/jpeg', 'image/png',
'text/plain'];
    const result = validateFileType(mockFile, allowedTypes);
    expect(result.valid).toBe(false);
    expect(result.error).toContain('Недопустимый тип файла');
  });
});
```

АТ-05:

```
const request = require('supertest');
const app = require('../../server');
const db = require('../../config/database');

let authToken;
let courseId;

beforeAll(async () => {
  const loginRes = await request(app)
    .post('/api/auth/login')
    .send({ email: 'autotest@mail.ru', password: 'hashedPassword123' });
  authToken = loginRes.body.token;

  // Создание курса и уроков для статистики
  const courseRes = await request(app)
    .post('/api/courses')
    .set('Authorization', `Bearer ${authToken}`)
    .send({
      title: 'Курс для статистики',
      description: 'Описание',
      startDate: '2025-01-01',
      endDate: '2025-12-31'
    });

  courseId = courseRes.body.course.id;

  // Создание 3 уроков
  for (let i = 1; i <= 3; i++) {
    await request(app)
      .post(`/api/courses/${courseId}/lessons`)
      .set('Authorization', `Bearer ${authToken}`)
      .send({
        title: `Урок ${i}`,
        deadline: '2025-06-01'
      });
  }

  // Отметить 2 урока как выполненные
  await request(app)
    .patch(`/api/courses/${courseId}/lessons/1/complete`)
    .set('Authorization', `Bearer ${authToken}`)
    .send({ completed: true });

  await request(app)
    .patch(`/api/courses/${courseId}/lessons/2/complete`)
    .set('Authorization', `Bearer ${authToken}`)
    .send({ completed: true });
});
```

```
describe('API Дашборд', () => {
  test('AT-05: Статистика дашборда рассчитывается корректно', async () => {
    const response = await request(app)
      .get('/api/dashboard/stats')
      .set('Authorization', `Bearer ${authToken}`);

    expect(response.status).toBe(200);
    expect(response.body).toHaveProperty('totalCourses');
    expect(response.body).toHaveProperty('completedLessons');
    expect(response.body).toHaveProperty('totalLessons');
    expect(response.body).toHaveProperty('progressPercent');

    // Проверка расчётов
    expect(response.body.totalCourses).toBeGreaterThanOrEqual(1);
    expect(response.body.completedLessons).toBe(2);
    expect(response.body.totalLessons).toBe(3);
    expect(response.body.progressPercent).toBeCloseTo(66.67, 1);
  });
});
```

№	ID теста	Название	Модуль	Статус	Время (ms)
1	AT-01	Вход с корректными данными	Авторизация	Успешно	150
2	AT-02	Валидация создания курса	Курсы	Успешно	200
3	AT-03	Отметка выполнения урока	Уроки	Успешно	250
4	AT-04	Валидация размера файла	Файлы	Успешно	50
5	AT-05	Расчёт статистики дашборда	Дашборд	Успешно	300

Заключение:

По результатам проведённого тестирования функциональность приложения SkillLvlUp реализована в соответствии с требованиями. Ключевой функционал (авторизация, управление курсами, отслеживание прогресса и экспорт) работает корректно.